Class / Pattern / Relationship Decision Checklist

A quick yes/no pass for any system, not just one project. Work top to bottom; each answer leads directly to an action.

---

1. Is it a class or an attribute? Does it have its own identity/lifecycle, separate from what it's attached to?

- **Yes → it's a class.** *A `Customer` in any e-commerce system — has an account, an ID, order history, referenced from many places.*
- **No → it's an attribute.** *"Loyalty tier" (bronze/silver/gold) — just a field on `Customer`, unless the tiers actually trigger different behavior somewhere.*

2. Does it need subclasses? Do the variants *behave* differently, not just hold different data?

- **Yes → make subclasses.** *In a ride-share app, `Car` vs. `Bike` — different fare calculation, different capacity rules.*
- **No → keep one class.** *`Car` in red vs. blue — that's a `color` field, not a reason to subclass.*

3. Should a class split into two? Do the parts follow one continuous sequence — does finishing one lead directly into the other starting?

- **Yes → keep it as one class, and use State.** A different actor triggering each stage does not by itself mean split. *A `JobApplication` — candidate submits, HR reviews and decides. Different people, same continuous lifecycle, not two objects.*
- **No → split them.** *An `Employee` class handling both payroll processing and office-supply ordering — same person doing both jobs, but no sequence connects "run payroll" to "order pens."*

4. Does it need a design pattern at all? If you wrote this the simplest possible way, would the exact same decision have to be written more than once, in more than one place?

- **No, only once → skip the pattern.** Plain code is fine. *A `Notification` class that only ever sends email, from one place in the code — nothing repeated, nothing to centralize.*
- **Yes, I'd have to write it again elsewhere → keep going below.** *Picking a payment type by `if type == "credit" / "debit" / "cash"` at checkout is plain code the first time — but if refund needs that exact same if/if/if to figure out which type to refund through, that's now two copies of the same decision. A pattern just means pulling it into one place both checkout and refund call, instead of maintaining two copies that can drift out of sync.*

5. Creational or Behavioral? Is this decision made repeatedly, at runtime, from more than one place?

- **Yes → check that table.** *Choosing a shipping carrier (FedEx/UPS/USPS) — decided at checkout, and again when generating a return label. Two decision points, same choice.*

- **No → check Structural next.** *A `Document`'s link to its renderer, assigned once when the document type is created — not a repeated runtime choice.*

6. Structural? Is this a fixed relationship between classes, true by design, not an event?

- **Yes → check that table.** *A UI toolkit wrapping each OS's native button behind one common `Button` interface — that wrapping relationship is just how the classes are built, always true.*

- **No → you likely don't need a pattern.** *A basic product-recommendation filter that just queries a list — no fixed structural relationship to speak of.*

7. What kind of relationship line? Does the other class die when this one is destroyed?

- **Yes → composition.** *An `Invoice` and its `LineItems` — a line item has no meaning outside the invoice it belongs to.*

- **No → does it store a permanent reference?**

  - **Yes → association.** *`Invoice` and `Product` — the product exists in the catalog whether or not this particular invoice references it.*
  - **No → dependency.** *`Invoice` creating a `TaxCalculator` to get one number, then discarding it — used once, never stored.*

8. Final check before locking it in Would deleting this line still leave a true, useful diagram?

- **Yes → cut it.** The diagram works fine without it, so it wasn't essential. *A leftover `Employee–Printer` line that doesn't reflect any behavior the system actually needs.*

- **No → keep it.** Removing it would lose something real. *`Order`→ `Customer` — a necessary, load-bearing fact about how the system works.*